APPANROM

A Tanglish Programming Language and Android App Development Studio

---

TEAM MEMBERS

P. Santhosh

M. Shalom Arputha Singh

Anand Institute of Higher Technology

The problem and our idea

Building a mobile app is hard to start for a beginner

The problem

- Native Android needs Kotlin or Java, Android Studio and Gradle knowledge
- Programming languages are written in English, a barrier for many first-time learners
- Designing screens and writing code are separate, disconnected steps
- Getting from source code to an installable APK needs a heavy toolchain

Our idea

- A small language with Tanglish keywords: mari, enna_na, ovvoru, screen_uruvakku
- A local-first web Studio: code editor, live preview and drag-and-drop designer
- One shared runtime, so the preview behaves exactly like the real app
- One click to generate an Android project and build an APK

What AppPanrom provides

Three parts that work together

1

Language

Tanglish keywords, indentation-based syntax, helpful error messages with suggestions. Compiled to a validated intermediate representation.

2

Studio

Web IDE with CodeMirror editor, live phone preview, visual designer, diagnostics, console and a build tab. Data stays on your machine.

3

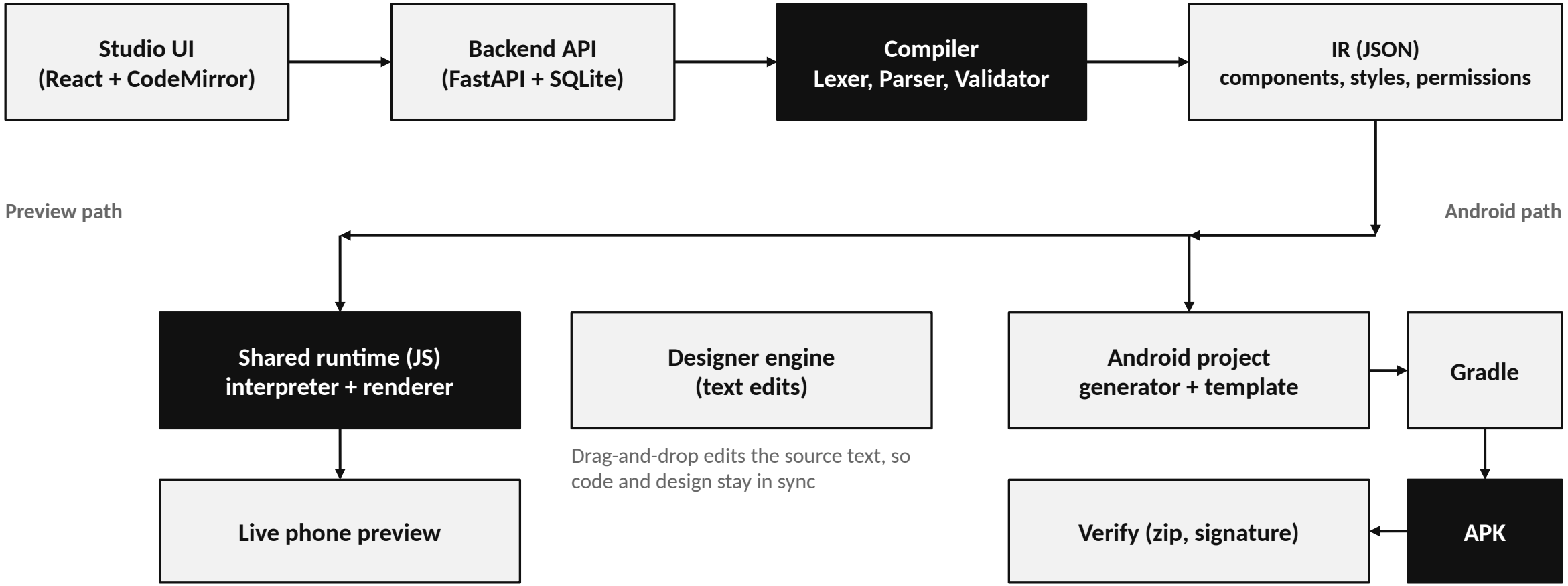
Android pipeline

Generates a real Gradle project from the same output, runs the build, verifies the APK and reports honestly.

Principles: no eval or exec of user code, secrets never stored in the repository, preview and APK share one runtime.

System architecture

One compiler output feeds both the preview and the Android app



The AppPanrom language

Readable Tanglish keywords with a familiar structure

```
app_uruvakku("Counter")

mari clicks = 0

function tap():
    clicks = clicks + 1
    enna_na clicks >= 10:
        toast_kaattu("Ten taps!")

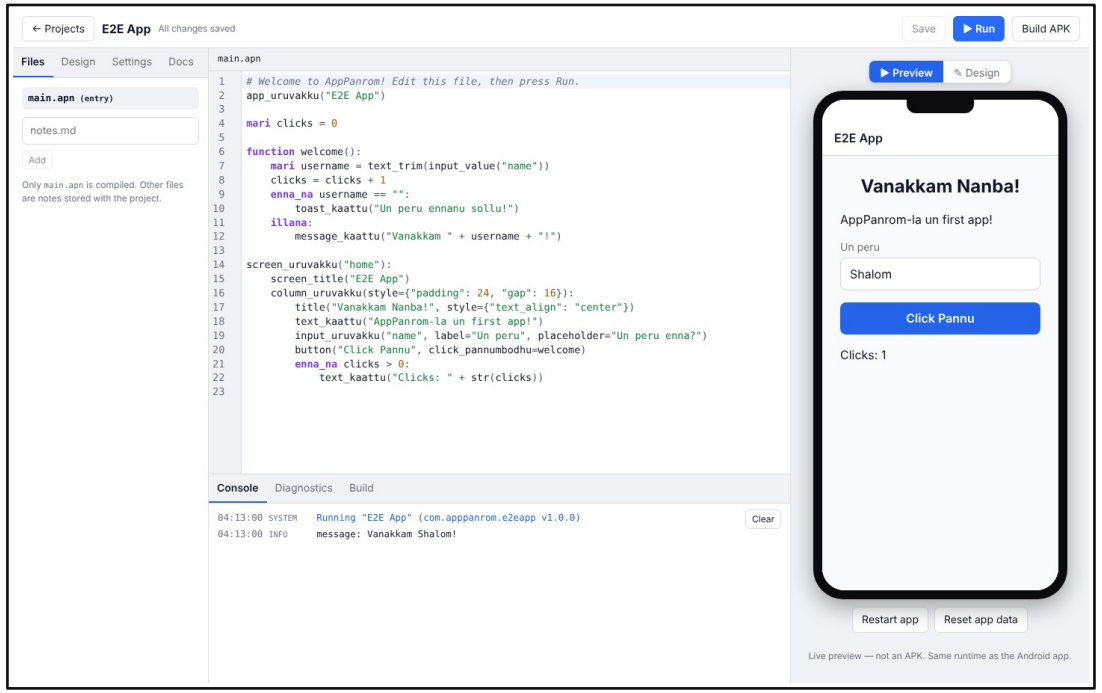
screen_uruvakku("home"):
    column_uruvakku():
        text_kaattu("Taps: " + str(clicks))
        button("Tap", click_pannumbodhu=tap)
```

`mari`	declare a variable
`enna_na / illana_enna / illana`	if / else if / else
`ovvoru ... in`	for-each loop
`irukkura_varai`	while loop
`screen_uruvakku`	define a screen
`click_pannumbodhu`	on-click handler

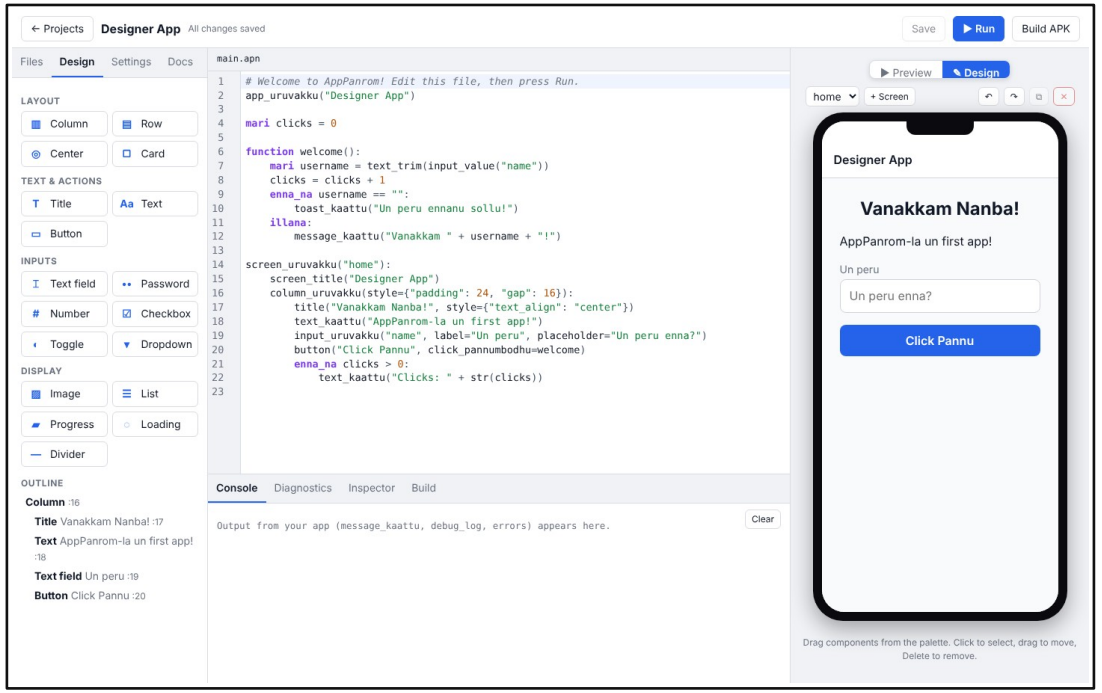
Hand-written indentation-aware lexer and parser. Errors come with line, column and a suggested fix. About 18 UI components and 50 built-in functions.

AppPanrom Studio

Write, preview and design in one window



Editor with live preview and console

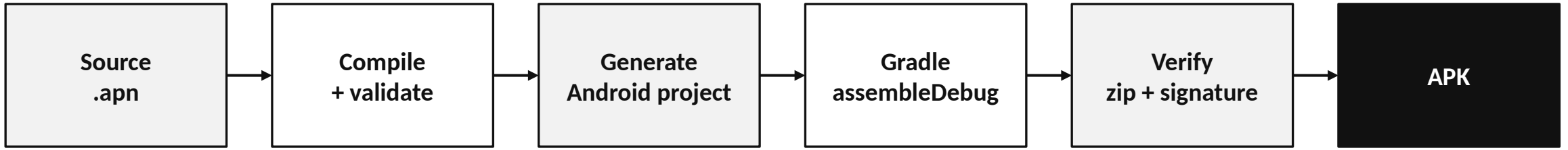


Visual designer: palette, outline, drag and drop

- Two-way sync: dragging a component rewrites the code, and typing code updates the canvas
- Undo and redo, inspector for styles, projects saved locally in SQLite

From source code to an installable APK

Honest reporting: a build succeeds only when a verified APK exists



Safe by design

Web content is served from local assets with a strict CSP. No JavaScript bridge. User code is interpreted, never evaluated.

Signing and permissions

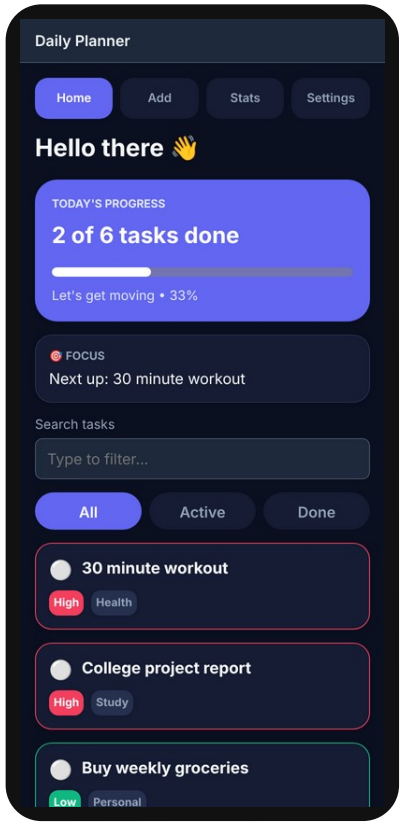
Release signing keys are read from environment variables only. Permissions are derived from the components and functions an app uses.

Built in CI

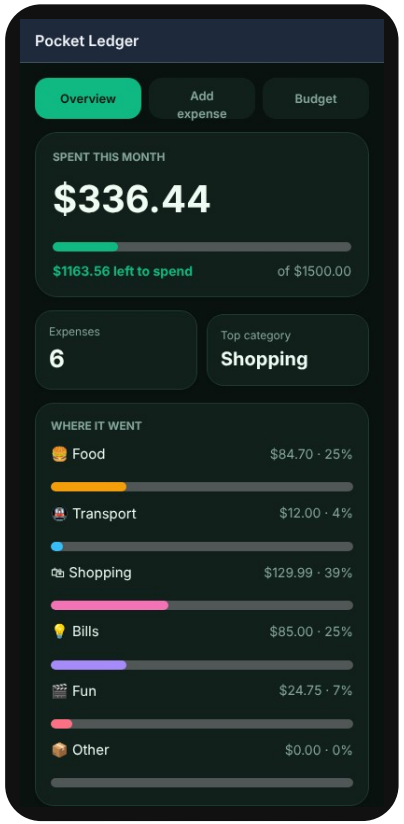
GitHub Actions builds each example with the Android SDK and Gradle, verifies the APK and publishes it with a SHA-256 checksum.

Apps built with AppPanrom

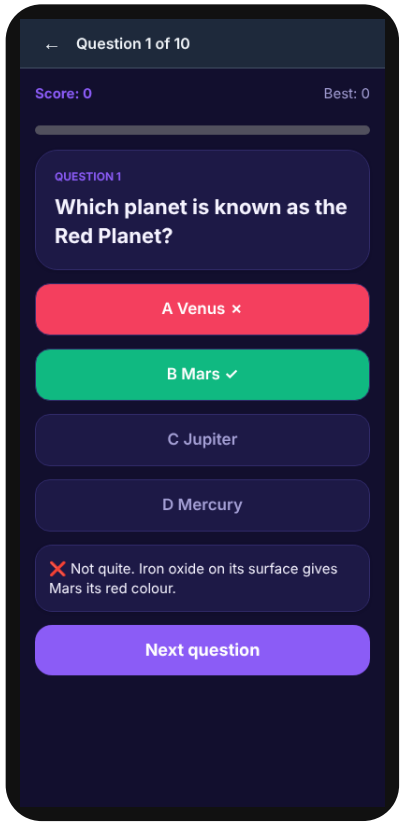
Seven example apps, all written in the language and built as real APKs



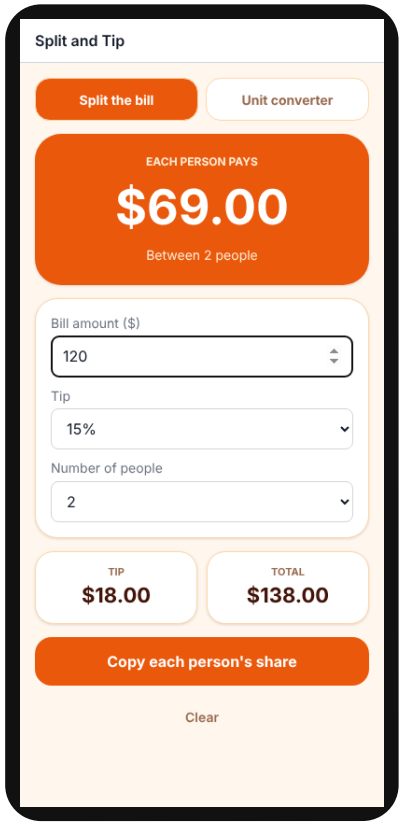
Daily Planner



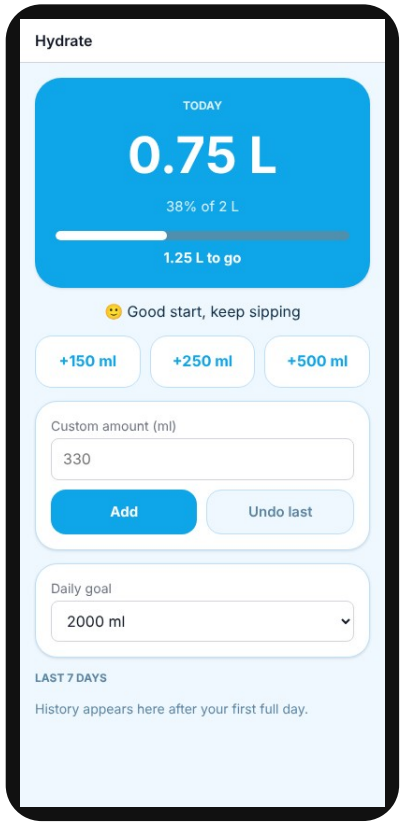
Pocket Ledger



Brain Quiz



Split & Tip



Hydrate

Also included: Calculator and the Tanglish "Vanakkam" starter. Screens are real renders of the shared runtime.

Results and conclusion

156
automated tests
(120 backend, 36 runtime)

7
example apps built
as real APKs

0
eval or exec of
user code

1
shared runtime for
preview and APK

Conclusion

AppPanrom turns a Tanglish program into a working Android app in one studio. The designer and code editor edit the same text, the preview and the APK run the same runtime, and every build is verified before it is reported as successful.

Next steps

- Test the APKs on a range of real devices
- Release signing and Play Store bundle (AAB) flow
- Network access, camera and location features